

Algorithms & Complexity — Խորը ուղեցույց

Big O, Binary Search, Vector/Map/Hash complexity

Այս ուղեցույցը մանրամասն բացատրում է algorithm complexity-ի հիմունքները՝ Big O notation, binary search, vector find, map insertion, hash table-ի աշխատանքը, օրինակներով և հարցազրույցի հարցերով:

1. Ի՞նչ է Big O Notation-ը

Big O-ն նկարագրում է algorithm-ի **արագության/հիշողության աճը** կախված input-ի չափից (n), երբ n-ը մեծանում է: Ցույց է տալիս **worst-case** վերին սահմանը:

$O(1)$	-> հաստատուն (input-ից անկախ)
$O(\log n)$	-> logarithmic (binary search)
$O(n)$	-> linear (մեկ loop)
$O(n \log n)$	-> merge/quick sort, efficient sort
$O(n^2)$	-> ներդրված loop (bubble sort)
$O(2^n)$	-> exponential (naive recursion)
$O(n!)$	-> factorial (permutations)

Աճի կարգ (արագ -> դանդաղ):

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

2. Big O օրինակներ

$O(1)$ — Հաստատուն

```
int x = arr[5];           // ուղիղ մուտք, input-ից անկախ
v.push_back(x);           // amortized  $O(1)$ 
```

$O(n)$ — linear

```
for (int i = 0; i < n; i++)    // n անգամ
    cout << arr[i];
```

$O(n^2)$ — ներդրված

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)    // n * n
        cout << arr[i][j];
```

$O(\log n)$ — կիսվում ամեն քայլ

```
while (n > 1)
    n = n / 2;    // ամեն քայլ կիսում -> log n
```

3. Binary Search Complexity

Binary search-ը **sorted** array-ում որոնում է, ամեն քայլ **կիսելով** որոնման տիրույթը:

```
int binarySearch(vector<int>& v, int target) {
    int l = 0, r = v.size() - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (v[mid] == target) return mid;
        else if (v[mid] < target) l = mid + 1;    // աջ կես
        else r = mid - 1;                        // ձախ կես
    }
    return -1;
}
```

Ամեն քայլ տիրույթը կիսում -> $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$
Քայլերի քանակ = $\log_2(n)$

Complexity. $O(\log n)$ (space $O(1)$): Պայման. array-ը պետք է **sorted** լինի:

```
n = 1000000 -> ~20 քայլ (log2(1000000))  
Linear search-ը կլինե՞ր 1000000 քայլ!
```

4. Vector-ում element գտնելը

find (unsorted) — $O(n)$

```
auto it = find(v.begin(), v.end(), target); //  $O(n)$  - ամեն element ստուգում
```

binary_search (sorted) — $O(\log n)$

```
sort(v.begin(), v.end()); //  $O(n \log n)$  մեկ անգամ  
binary_search(v.begin(), v.end(), x); //  $O(\log n)$ 
```

index-ով access — $O(1)$

```
int x = v[5]; //  $O(1)$  - ուղիղ
```

Vector search:

unsorted find	-> $O(n)$
sorted binary	-> $O(\log n)$
index access	-> $O(1)$

5. Map-ում insertion

std::map (Red-Black Tree)

```
map<int, string> m;  
m[5] = "hello"; //  $O(\log n)$  - tree-ի մեջ տեղ գտնել + balance  
m.insert({3, "x"}); //  $O(\log n)$ 
```

$O(\log n)$ -- balanced tree-ի մեջ տեղ գտնելը logarithmic է:

std::unordered_map (Hash Table)

```
unordered_map<int, string> m;  
m[5] = "hello";           // O(1) average, O(n) worst
```

O(1) average (hash), **O(n) worst** (collision/rehash):

```
map          insert -> O(log n)      (sorted, RB-tree)  
unordered_map insert -> O(1) average (hash)
```

6. Hash Table ինչպես է աշխատում

Hash table-ը key-ը անցկացնում է **hash function**-ի միջով -> ստանում **bucket index** -> element պահվում այդ bucket-ում:

```
key --> hash(key) --> index --> bucket[index]
```

```
hash("apple") = 3 -> bucket[3]
```

```
hash("banana") = 7 -> bucket[7]
```

Lookup

1. hash(key) հաշվել -> O(1)
2. bucket գտնել -> O(1)
3. bucket-ում element գտնել -> O(1) average (քիչ collision)

Collision

Երբ երկու key նույն bucket-ն են ստանում:

```
hash("cat") = 3  
hash("dog") = 3    <- collision  
bucket[3]: (cat) -> (dog)  [chaining - linked list]
```

Շատ collision -> bucket-ի list երկարում -> lookup O(n) դառնում:

Load factor & rehash

load factor = element-ներ / bucket-ներ

Գերազանցելիս (~ 1) $\rightarrow$ rehash: bucket-ներ $\times 2$, բոլոր element վերաբաշխում ($O(n)$)

7. Complexity Cheat (STL container-ներ)

Container	Access	Search	Insert	Delete
vector	$O(1)$	$O(n)$	$O(1)^*$ end	$O(n)$ mid
list	$O(n)$	$O(n)$	$O(1)$	$O(1)$
map/set	-	$O(\log n)$	$O(\log n)$	$O(\log n)$
unordered_map/set	-	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
stack/queue	-	-	$O(1)$	$O(1)$
priority_queue	$O(1)$ top	-	$O(\log n)$	$O(\log n)$

* amortized.

8. Հարցազրույցի հարցեր և պատասխաններ

Big O ի՞նչ է:

Algorithm-ի արագության/հիշողության աճը input-ի չափից (n); worst-case վերին սահման:

Binary search complexity:

$O(\log n)$ - ամեն քայլ տիրույթը կիսում. Պայման: sorted:

Vector-ում element գտնել:

unsorted find $O(n)$; sorted binary_search $O(\log n)$; index access $O(1)$:

Map insert complexity:

std::map $O(\log n)$ (RB-tree); unordered_map $O(1)$ average (hash):

Hash table ինչպես:

hash(key) -> bucket index -> պահել/գտնել. $O(1)$ average, $O(n)$ worst (collision):

Ինչո՞ւ binary search $O(\log n)$:

Ամեն քայլ input-ը կիսում ($n \rightarrow n/2 \rightarrow \dots \rightarrow 1$) -> $\log_2(n)$ քայլ:

$O(n \log n)$ ինչ algorithm:

Efficient sort (merge, quick, heap):

map vs unordered_map complexity:

map $O(\log n)$ sorted; unordered_map $O(1)$ avg unsorted:

Ամփոփում (Cheat Sheet)

BIG O (արագ -> դանդաղ):

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

BINARY SEARCH: $O(\log n)$, sorted array, ամեն քայլ կիսում

$n=1M \rightarrow \sim 20$ քայլ

VECTOR SEARCH: unsorted $O(n)$ | sorted binary $O(\log n)$ | index $O(1)$

MAP INSERT: map $O(\log n)$ (RB-tree) | unordered_map $O(1)$ avg (hash)

HASH TABLE: hash(key) -> bucket -> $O(1)$ avg, $O(n)$ worst (collision)

load factor $> \sim 1 \rightarrow$ rehash ($\times 2$ buckets, $O(n)$)

STL: vector access $O(1)$ | map $O(\log n)$ | unordered $O(1)$ avg | heap top $O(1)$